WEEK 2 · LESSON 10 OF 24 · HANDOUT

Data Drift and Concept Drift

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=0wdCMvU1vCQ — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2021: Production ML Monitoring — Outliers, Drift, Explainers and Statistical Performance — https://www.youtube.com/watch?v=n0bR0IArJDo

🎓 **Go deeper:** PyCon DE 2023: Staying Alert — How to Implement Continuous Testing for Machine Learning Models — https://www.youtube.com/watch?v=4BgmBtsS0iA

Your goal in this lesson: Measure data drift with PSI and the KS test, understand concept drift, and see why you need both input monitoring and outcome monitoring.

The world does not stand still

- **Data drift:** the inputs change. Prices rise, so loans get bigger; a new mobile-money product changes transaction counts.
- **Concept drift:** the **link** between inputs and outcome changes. A large employer closes, so "has a salary" no longer protects against default.

A model trained on last year can be wrong this year in either case.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

Measuring drift: PSI

The **Population Stability Index** compares how a feature is spread across bins in the training data and in new data:

> *PSI = sum over bins of (new% − old%) × ln(new% ÷ old%)*

A common rule of thumb: **below 0.1** no real change, **0.1–0.25** some change (watch), **above 0.25** big change (investigate).

```
from scipy.stats import ks_2samp

def psi(reference, current, bins=10):
    edges = np.unique(np.quantile(reference, np.linspace(0, 1, bins + 1)))
    edges[0], edges[-1] = -np.inf, np.inf
    ref = np.histogram(reference, edges)[0] / len(reference)
    cur = np.histogram(current, edges)[0] / len(current)
    ref, cur = np.clip(ref, 1e-4, None), np.clip(cur, 1e-4, None)
    return float(np.sum((cur - ref) * np.log(cur / ref)))

def drift_report(reference, current):
    rows = []
    for f in FEATURES:
        value = psi(reference[f], current[f])
        rows.append({"feature": f, "psi": round(value, 3),
                     "ks_p_value": round(ks_2samp(reference[f], current[f]).pvalue, 4),
                     "status": "BIG CHANGE" if value > 0.25 else "watch" if value > 0.1 else
"ok"})
    return pd.DataFrame(rows)

same_world = make_applicants(n=1000, seed=21)
later = make_applicants(n=1000, seed=22, drift=0.4)
print("new data from the same world:"); print(drift_report(train,
same_world).to_string(index=False))
print("\na year later (prices and late payments up):"); print(drift_report(train,
later).to_string(index=False))
```

The **KS test** asks "could these two samples come from the same distribution?"; a tiny p-value says no. Look at income in the first table: the KS test gives p ≈ 0.03 although the data comes from the **same** world — a false alarm. With large samples it flags even unimportant changes, so PSI's size is usually more useful for decisions.

Does data drift hurt the model?

```
def evaluate(data):
    p = model.predict_proba(data[FEATURES])[:, 1]
    return {"accuracy": round(accuracy_score(data[TARGET], p >= 0.5), 3),
            "auc": round(roc_auc_score(data[TARGET], p), 3), "approve_rate": round(float((p >=
0.5).mean()), 3)}

print("same world:", evaluate(same_world))
print("a year later:", evaluate(later))
```

Surprise: the inputs moved a lot (loan size PSI far above 0.25) but accuracy is **about the same**. The model still applies the same, correct rules — it just approves fewer people because loans really are riskier. Data drift is an **alarm to investigate**, not automatic proof that the model is broken.

Concept drift: the silent killer

Now the rules of the world change, but the inputs look the same:

```
changed = make_applicants(n=1000, seed=21, concept=1.5)
print("max PSI:", drift_report(train, changed)["psi"].max())
print("same world:   ", evaluate(same_world))
print("concept drift:", evaluate(changed))
```

PSI sees **nothing**, the approval rate is **identical** — but accuracy falls by about **12 points** (from 0.82 to 0.70). Only one thing reveals concept drift: comparing predictions with **real outcomes** when they arrive.

A monitoring plan that covers both

1. **Inputs:** PSI per feature every week; investigate above 0.25.
2. **Outputs:** approval rate and probability distribution every day.
3. **Outcomes:** as repayments come in, accuracy and AUC per month — the only defence against concept drift.
4. **Early outcome signals:** in lending, "missed first payment" arrives sooner than "fully repaid".
5. **Response:** investigate → retrain on recent data → test → promote through the registry (Lesson 8).

Key terms

- **Data drift** — input distributions change.
- **Concept drift** — the relationship between inputs and outcome changes.
- **PSI** — Population Stability Index, a size of distribution change.
- **KS test** — a statistical test for whether two samples have the same distribution.
- **Label delay** — the time until real outcomes are known.

Remember this: Measure input drift with PSI, but remember that data drift does not always hurt and concept drift can hurt without any data drift — so always monitor real outcomes too.

Name: _____ Date: _____

Activity

Task 1

Compute PSI for `drift=0.1`, `0.2` and `0.3`. At which value does loan size cross 0.25?

Task 2

Try `concept=0.5` and `concept=1.0`. How much accuracy is lost?

Task 3

Retrain the model on 2,000 rows of `make_applicants(seed=30, concept=1.5)` and evaluate it on `changed`.

Task 4

Write a short message to the SACCO manager explaining concept drift in three sentences.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is data drift?
   - A. The distribution of inputs changes
   - B. Accuracy improves
   - C. Data is deleted
   - D. The model file moves
2. What is concept drift?
   - A. A slow server
   - B. Inputs change colour
   - C. A bug in pandas
   - D. The relationship between inputs and outcome changes
3. By a common rule of thumb, what does PSI above 0.25 mean?
   - A. Delete the data
   - B. A big change: investigate
   - C. The model is perfect
   - D. No change
4. In the lesson, what happened to accuracy under strong data drift?
   - A. It reached 100%
   - B. It collapsed
   - C. It stayed about the same
   - D. It became negative
5. Why is concept drift called silent?
   - A. It deletes logs
   - B. It makes no noise
   - C. Inputs and approval rate can look unchanged while accuracy falls
   - D. It only happens at night
6. What is the only reliable way to detect concept drift?
   - A. Counting requests
   - B. PSI
   - C. Comparing predictions with real outcomes when they arrive
   - D. Restarting the server

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.